

Project 07 — Online Clothing Brand

Course: Web Technologies | **Format:** Project | **Duration:** 1 week

Application Overview

An online shopping website for a clothing brand where customers can view products, browse by gender and category, add items to cart, and place orders. Two types of users: **ADMIN** (owner) and **CUSTOMER**.

- **ADMIN:** primary control – adds/deletes/updates products and details, deletes existing customers, views order list and confirms/rejects orders, views all purchase history of every customer.
- **CUSTOMER:** registered user – browses products by gender and category, views images, descriptions, size charts, adds to cart, removes cart items, views invoice, selects payment method, views purchase history in profile.
- **Unregistered users** can browse but cannot make purchase requests.

Team Structure & Student IDs

Student ID	Task	Main Features
	Task 1	User Authentication (Admin/Customer), Profile Management, Home Page with Gender/Category Navigation, Basic Browsing
	Task 2	Admin – Product Management (CRUD), Delete Customers, Order List (confirm/reject), View All Purchase History
	Task 3	Customer – Search & Filtering (AJAX), Product Details (images, description, size chart), Add to Cart, Cart Management (add/remove/update)
	Task 4	Customer – Checkout, Invoice, Payment Method Selection, Order Finalization, Purchase History in Profile, Admin Order Processing (confirm/reject)

Shared Database Schema (same for all)

Table	Key Columns
users	id, name, email, password_hash, role (admin/customer), profile_picture, address, phone, created_at
categories	id, name, parent_id (for hierarchical: Gender → Clothing type), created_at
products	id, name, description, size_chart (text or JSON), price, category_id, image_path, stock, gender (Men/Women), created_at
cart	id, user_id, product_id, quantity, added_at
orders	id, user_id, total_amount, status (pending/confirmed/rejected), order_date
order_items	id, order_id, product_id, quantity, unit_price
payments	id, order_id, amount, payment_method, transaction_id, payment_date

No code sharing except the schema.

Global Technical Requirements

- PHP MVC (controllers/, models/, views/, config/)
- Passwords hashed with `password_hash()` / verified with `password_verify()`
- PDO/mysqli + **prepared statements** for every query – no string-concatenated SQL
- Server-side validation on every form before any DB write; show inline error messages
- `session_start()` on every page that requires auth; redirect unauthenticated users
- AJAX endpoints return `Content-Type: application/json`; handle errors client-side
- File uploads go to `public/uploads/` (profile pictures, product images) – validate MIME type + size server-side
- Do not drop or alter the shared schema tables

Mandatory Grading Criteria (each student must satisfy all 10 within their task)

1. **Basic Web Security** – SQL injection prevention, XSS protection, CSRF awareness, secure password storage.
2. **UI (HTML/CSS)** – Clean, responsive, user-friendly interface.
3. **Feature Completeness** – All assigned requirements work error-free.
4. **DB** – Correct use of shared schema, proper relationships, data integrity.
5. **Auth (Session/Cookie)** – Session management, role-based access, “Remember Me” (optional).
6. **MVC** – Clear separation of business logic, presentation, and request handling.
7. **JS Validation** – Client-side validation on forms (e.g., registration, product upload, checkout).
8. **PHP Validation** – Server-side validation on every input before DB write.
9. **Ajax/JSON** – At least one AJAX endpoint returning JSON per student.
10. **Git Contribution** – Feature branches, meaningful commits (≥ 3 per student), merge into main via PR.

Git Flow

- Repository has a **main** branch (protected, no direct pushes).
- Each student creates a **feature branch** from **main** named **feature/taskX-studentID** (e.g., **feature/task1-2352573-2**).
- Work on the feature branch, commit regularly (at least 3 commits per student).
- After completing the feature, open a **pull request** into **main** and merge.
- Final **main** must contain all four feature branches merged with full history.

Task Breakdown

Task 1 – User Authentication (Admin/Customer), Profile Management, Home Page with Gender/Category Navigation, Basic Browsing

Student:

CRUD included: Create (registration), Read/Update (profile).

Requirements:

- **Registration** – form for ADMIN and CUSTOMER. Collect: name, email, password (≥ 8 chars), address, phone. Role dropdown. Validate unique email, hash password. Redirect to login.
- **Login & Remember Me** – login creates `$_SESSION['user_id']`, `$_SESSION['name']`, `$_SESSION['role']`. “Remember Me” cookie + token for auto-login.

- **Profile Page** (session-gated) – both admin and customer can update name, email, address, phone, profile picture, change password (require current). Show success banner.
- **Logout & Navbar** – destroy session and cookie. Navbar shows links based on role.
- **Home Page** – shows:
 - Gender-based navigation (Men, Women) – each leading to categories for that gender.
 - Featured products (latest or random).
 - Search box (UI only, functionality in Task 3).
- **Basic Browsing** – customers (and unregistered users) can browse categories:
 - For Men: Shirts, Pants, etc. For Women: Salwar, Jeans, etc.
 - Clicking a category shows all products under that category (view-only).
- **JS validation** on registration and profile forms.

Key outputs: users table with admin and customer accounts; session management; home page with gender and category navigation; basic browsing of products.

Task 2 – Admin: Product Management (CRUD), Delete Customers, Order List (confirm/reject), View All Purchase History

Student:

CRUD included: Full CRUD on products + Delete on users (customers) + Read/Update on orders.

Requirements:

- **Admin Gate** – every admin page checks `$_SESSION['role'] === 'admin'`; redirect others.
- **Product Management** – create, edit, delete products.
 - Form: name, description, size chart (text area or JSON), price (>0), gender (Men/Women), category (dependent on gender), image upload (JPEG/PNG ≤2MB to public/uploads/products/), stock.
 - Edit pre-fills form.
 - Delete removes image file and product (block if product is in any order? Or set null).
- **Delete Customer** – list all customers (role='customer') with delete button. Cascade remove customer's cart, orders, etc.
- **Order List** – admin page showing all orders with: customer name, total amount, status (pending/confirmed/rejected), order date. Admin can **confirm** or **reject** each order (update status field via AJAX or form).
- **View All Purchase History** – separate admin page showing every customer's purchase history (orders, items, quantities, total).
- **Admin Dashboard** – summary counts (total products, total customers, total orders, pending orders).

- **PHP & JS validation** on product forms (price positive, image type).
- **AJAX** – for confirm/reject order status update without page reload.

Key outputs: Admin fully manages product inventory, deletes customers, views orders, confirms/rejects orders, and sees all purchase history.

Task 3 – Customer: Search & Filtering (AJAX), Product Details, Add to Cart, Cart Management

Student:

CRUD included: Create, Update, Delete on `cart` + Read for search.

Requirements:

- **Customer Gate** – cart operations require login; browsing/search is public.
- **Search Box & Filtering (AJAX)** – on home page, search box with filtering options:
 - Search by product name, or filter by category (e.g., Shirts, Pants, Salwar, Jeans) or gender.
 - AJAX GET `/api/products/search?q=&category=&gender=` returns JSON, JS dynamically updates product grid.
- **Product Details Page** – separate page for each product (`/product/{id}`). Shows:
 - Images (multiple if needed), short description, size chart, price, stock, gender, category.
 - “Add to Cart” button.
- **Add to Cart** – AJAX POST to `/api/cart/add` with `product_id` and `quantity`. Updates DB cart (or session). Returns new cart count. JS updates cart icon in navbar.
- **Cart Page** – list cart items (product name, price, quantity, subtotal).
 - Options: remove item (AJAX DELETE), update quantity (+/-) via AJAX.
 - Show total cart value.
 - Customer can browse other pages and add more items; cart persists.
- **JS validation** – search filters, cart quantity (positive integer, not exceeding stock).
- **PHP validation** – product exists, stock available.

Key outputs: Customers can search with filters, view detailed product information, add/remove/update cart items across sessions.

Task 4 – Customer: Checkout, Invoice, Payment Method Selection, Order Finalization, Purchase History in Profile, Admin Order Processing (confirm/reject)

Student:

CRUD included: Create (order, order_items, payment), Read (purchase history), Update (order status – but admin part overlaps Task2; here include admin confirm/reject if not done). Actually the requirement says “Only the ADMIN will be able to view the order list and confirm or reject it.” So Task 2 handled order list and confirm/reject. Task 4 will focus on customer checkout, invoice, payment, purchase history. But to avoid missing admin’s confirm/reject in Task4, we note it’s already in Task2. We’ll keep Task4 as customer finalization and purchase history, plus admin can view order list (already Task2). Ensure no duplication.

Requirements:

- **Customer Gate** – checkout requires login.
- **Checkout from Cart** – from cart page, customer clicks “Proceed to Checkout”.
 - Show invoice summary: list of products, quantities, unit price, total amount.
 - Two buttons: **Cancel** (back to cart) and **Continue**.
 - If Continue, show payment method selection (Credit Card, bKash, Nagad, Bank Transfer, Cash on Delivery).
 - On final submit: validate cart not empty, create orders row (status='pending'), create order_items rows, store payment info (or simulate), clear cart.
 - Redirect to order confirmation page showing order ID.
- **Purchase History in Customer Profile** – in profile page, a section showing all past orders (order ID, date, total amount, status, link to details). Detailed list of items per order.
- **Admin Order Processing** (already in Task2) – but to ensure completeness, Task4 will also include that the admin can view order list and confirm/reject (may be a shared responsibility or duplicated). To avoid conflict, Task2 handles admin confirm/reject; Task4 focuses on customer side. We’ll state that Admin’s confirm/reject is in Task2, and Task4 provides the order data that admin acts upon.
- **JS validation** – checkout (address required, payment method selected).
- **AJAX** – for placing order without page reload (optional).

Key outputs: Customers can finalize purchase, see invoice, choose payment method, view detailed purchase history in profile. Admin (Task2) can confirm/reject orders.